

Pavbot Intelligence - System AI Automation End-to-End

System end-to-end: agenci, research, dane, manifest, iOS, backend, powiadomienia i release.

Główna teza

Pavbot jest kompletnym systemem automatyzacji AI, a nie pojedynczym promptem. Łączy agentic workflows, walidowane artefakty, natywną aplikację iOS, backend powiadomień APNs oraz proces jakości i wydania.

ZAKRES

AI workflow + iOS + backend

ARTEFAKTY

JSON, PDF, audio, manifest

JAKOŚĆ

Swift/Python tests + verifier

RELEASE

App Store archive 2.0

01

Perspektywa techniczna

Z perspektywy roli AI/Automation Engineer najmocniejszy sygnał to integracja AI z klasyczną inżynierią: automatyzacja tworzy dane, ale system ma kontrakty, cache, testy, diagnostykę, deployment i przewidywalny release pipeline.

Najmocniejszy sygnał

Potrafię zaprojektować workflow, w którym agenci tworzą użyteczne artefakty, a aplikacja i backend konsumują je stabilnie i powtarzalnie.

Dlaczego jest wiarygodne

Repo zawiera iOS app, FastAPI notifier, manifest generator, walidatory, testy, dokumentację operacyjną i App Store archive 2.0.

Najlepszy angle rozmowy

Mówić o produkcyjnym cyklu danych: źródła -> research -> JSON/PDF/audio -> manifest -> mobile UX -> push notifications.

Granice narracji

Nie przedstawiać tego jako skalowanego SaaS z metrykami użytkowników. To osobisty system z realnym workflow i dojrzałą strukturą.

Czym jest Pavbot

Lokalno-chmurowy system automatyzacji researchu, który zamienia cykliczne przebiegi Codex w uporządkowane raporty, dane JSON, PDF-y, podcasty i natywne ekrany iOS.

Problem automatyzacyjny

Monitoring newsów, ofert AI/LLM, raportów i podcastów jest powtarzalny, rozproszony i podatny na utratę kontekstu. Pavbot robi z tego kontrolowany pipeline.

Mechanika danych

Automatyzacje publikują artefakty do repo, generator tworzy publiczny manifest, a aplikacja wybiera strukturalne dane typu jobsData, researchData, pulseNewsData albo fallback PDF/Markdown.

Jakość działania

Projekt ma testy Swift/Python, verifier workspace, cache, fallbacki, diagnostykę połączeń, blokadę produkcyjnych URL-i i powtarzalny proces archive.

Manifest jako kontrakt

Aplikacja nie zna szczegółów każdego przebiegu automatyzacji. Czyta manifest i wybiera najlepszy artefakt dla widoku.

Natywne renderowanie

JSON jest zamieniany na ekrany iOS, a PDF/Markdown zostają jako fallback dla starszych lub nietypowych artefaktów.

Operacyjna odporność

Cache, diagnostyka, testy i verifier zmniejszają ryzyko, że automatyzacja opublikuje coś nieużytecznego dla aplikacji.

Konfiguracja produkcyjna

Wersja 2.0 wymusza produkcyjne URL-e i usuwa możliwość przypadkowego przestawienia aplikacji na stare linki.

04

Obszary inżynierskie

Agentic workflows	Prompt/skill design	Data validation	Manifest-driven UX
SwiftUI	FastAPI	APNs	Caching
Fallback design	XcodeGen	Release pipeline	Operational docs

05

Techniczne dowody jakości

- Natywna aplikacja iOS z zakładkami Today, Pulse Day, Jobs, Research, audio/TTS, diagnostyką i ustawieniami.
- Backend Pavbot Notifier obsługuje GitHub webhook, manifest diffing, rejestrację urządzeń i APNs.
- Artefakty publikowane są przez manifest zamiast ręcznego kopiowania danych do aplikacji.
- Wersja 2.0 wymusza produkcyjne URL-e i ignoruje stare lokalne ustawienia użytkownika.
- Walidacja obejmuje testy iOS, testy Python, workspace verifier, diff check oraz archive App Store Connect.

06

Jak o tym mówić

30 sekund

Pavbot to system AI automation end-to-end. Agenci Codex wykonują research, generują strukturalne dane, PDF-y i audio, manifest publikuje wyniki, a natywna aplikacja iOS pokazuje je z cache, fallbackami i powiadomieniami.

2 minuty

Najważniejsze w Pavbot jest to, że AI nie kończy się na odpowiedzi w czacie. Zbudowałem pipeline, w którym automatyzacje mają kontrakty tematów, zapisują raporty i dane, generator tworzy manifest, backend wykrywa nowe publikacje i może wysyłać APNs, a aplikacja iOS renderuje wyniki jako produkt. To pokazuje projektowanie agentów, integrację systemów, walidację danych, mobile UX i operacyjne myślenie o jakości.